Trabalho Final de Compiladores

Djonatan R. C. Bonelli[1], Andrei C. Camilotto[2]

Ciência da Computação – Universidade Federal da Fronteira Sul (UFFS)
Caixa Postal 181 – 89815-899 – Chapecó– SC – Brazil

djonatan.bonelli@estudante.uffs.edu.br,
andrei.camilotto@estudante.uffs.edu.br

Abstract. *This article explores the fundamental stages of the compilation process for programming languages. A compiler converts source code into executable machine code, making it essential for software development. Key phases include lexical analysis, syntax analysis, semantic actions, optimization, and code generation. Lexical analysis produces tokens; syntax analysis builds the parse tree; semantic actions verify the logical consistency of some of these specific parts of the code; optimization enhances performance; and code generation transforms the tree into machine code. This article provides a clear overview of these stages, explaining their functions and interactions.*

Resumo. *Este artigo explora as etapas fundamentais do processo de compilação de linguagens de programação. Um compilador converte código-fonte em código de máquina executável, sendo essencial para o desenvolvimento de software. As fases principais incluem: análise léxica, análise sintática, ações semânticas, otimização e geração de código. A análise léxica gera tokens; a análise sintática constroi a árvore de derivação; as ações semânticas verificam a consistência lógica de partes específicas do código; a otimização melhora o desempenho; e a geração de código transforma a árvore em código de máquina. O artigo oferece uma visão geral de algumas dessas etapas, explicando suas funções e interações.*

1. Introdução

Os compiladores desempenham um papel fundamental no desenvolvimento de software moderno, atuando como intermediários entre linguagens de programação de alto nível e o código de máquina executável. Seu principal propósito é traduzir programas escritos em linguagens como Java, C ou Python para uma forma que o hardware do computador possa entender e executar. Além de tradução, os compiladores também realizam otimizações, verificações de erros e, em muitos casos, oferecem feedback ao programador durante o processo de desenvolvimento.

Com a crescente complexidade dos sistemas de software e a diversidade de linguagens de programação, os compiladores se tornaram ferramentas essenciais em diversos domínios, desde o desenvolvimento de aplicativos até sistemas embarcados e jogos eletrônicos. As características dos compiladores incluem a capacidade de detectar erros

de sintaxe e semântica, otimizar o desempenho do código gerado e fornecer uma interface clara para os desenvolvedores.

Um dos principais problemas enfrentados na construção de compiladores é a necessidade de balancear a eficiência da tradução com a legibilidade e a manutenção do código-fonte original. Além disso, com a evolução constante das linguagens e das arquiteturas de hardware, os compiladores precisam se adaptar às novas exigências do mercado, e portanto, este trabalho tem como objetivo explorar os fundamentos dos compiladores, abordando suas etapas de funcionamento.

2. Analizador Léxico

O analisador léxico, também conhecido como lexer ou scanner, é a primeira etapa do processo de compilação. Sua função é ler o código-fonte como uma sequência de caracteres e transformá-lo em uma sequência de tokens, que são unidades sintáticas básicas, como palavras-chave, identificadores, operadores e delimitadores. A análise resulta em uma fita de saída, em que cada token gerado é associado a uma categoria, que será usada nas etapas subsequentes da compilação, facilitando a análise sintática e semântica.

A análise é feita com o uso de um autômato finito determinístico (AFD) para identificar palavras reservadas e separar a cadeia de entrada em tokens válidos para a linguagem utilizada. O AFD lê a cadeia de entrada até encontrar um caractere considerado como separador, nesse ponto ele identifica o token reconhecido e, se necessário, ele também define um token para o separador em questão, em seguida reinicia o processo de reconhecimento a partir da posição seguinte do separador.

2.1. Implementação

A implementação feita foi baseada em um autômato finito determinístico (AFD), do qual foi adaptado para o código python em um dicionário de transições entre estados. Este funciona da seguinte forma: O arquivo com o texto é aberto e vira uma string, nela, começaremos a ler o primeiro símbolo a partir do estado zero do autômato, e para cada caractere seguinte, buscamos por uma transição daquele estado com aquele símbolo até encontrarmos um separador, quando isso ocorre, salvamos na fita de saída a string e o token de acordo com o estado em que ele parou, além disso, lemos o separador também para adicioná-lo à fita, já que temos operadores que servem de separadores, logo, também são tokens válidos. Após salvar o separador, voltamos ao estado zero para começar a reconhecer o próximo token.

Caso o estado final do autômato seja o estado de erro, adicionamos “ERRO” à fita de saída junto com a string lida, e o mesmo vale para caso encontremos um caractere inválido durante a leitura de um token, quando não há saída daquele estado para qualquer outro estado a partir daquele símbolo, que é equivalente à uma transição implícita para o estado de erro.

A tabela abaixo identifica os tokens utilizados:

TOKEN	Valor léxico
SE	‘se’
SENAO	‘senao’
ENQUANTO	‘enquanto’
VAR	^[a-zA-Z][a-zA-Z0-9]*$
NUM	^[0-9]+$
ERRO	^[0-9]+[a-zA-Z]+$
ATRIB	‘=’
MAIS	‘+’
MENOS	‘-’
MULTI	‘*’
DIV	‘/’
PARE	‘(’
PARD	‘)’
CHAE	‘{’
CHAD	‘}’
PVIRGULA	‘;’
MAIOR	‘>’
MENOR	‘<’
DIF	‘!’
BRANCO	‘\t’, ‘ ‘, ‘$’
LINHA	‘\n’

3. Analisador Sintático

Os analisadores sintáticos, também conhecidos como parsers, são componentes fundamentais dos compiladores, responsáveis por interpretar a estrutura gramatical do código-fonte para validar sua conformidade com a linguagem de programação. Eles leem a saída do analisador léxico (fita de tokens) e organizam esses tokens em uma estrutura de dados hierárquica, como uma árvore de análise (ou árvore sintática).

Um dos métodos mais eficientes de análise sintática é a análise LR (Left-to-right, Rightmost derivation), que lê a entrada da esquerda para a direita, construindo a derivação mais à direita. Compiladores LR são bastante utilizados porque lidam bem com gramáticas de linguagens complexas e oferecem alta performance para linguagens de programação de uso geral. Esses analisadores são divididos em variações como LR(0), SLR(1), LALR(1) e LR(1), cada uma com técnicas e capacidades específicas para balancear precisão e eficiência, e sendo portanto, muito usados em compiladores modernos

3.1 Implementação

A implementação realizada é baseada em um analisador sintático do tipo SLR, que processa uma Gramática Livre de Contexto (GLC) através de uma tabela de parsing. A GLC processada é definida abaixo:

1. S → Prog
2. Prog → Decl Prog
3. Prog → Cond Prog
4. Prog → Loop Prog
5. Prog → ε
6. Decl → var = Expr ;
7. Expr → Expr + Termo
8. Expr → Expr - Termo
9. Expr → Termo
10. Termo → Termo * Fator
11. Termo → Termo / Fator
12. Termo → Fator
13. Fator → (Expr)
14. Fator → num
15. Fator → var
16. Cond → se (Vali) Bloco
17. Cond → se (Vali) Bloco senao Bloco
18. Loop → enquanto (Vali) Bloco
19. Vali → Expr = = Expr
20. Vali → Expr > Expr
21. Vali → Expr < Expr
22. Vali → Expr > = Expr
23. Vali → Expr < = Expr
24. Vali → Expr ! = Expr
25. Vali → (Vali)
26. Bloco → { Prog }

O código conta com dois dicionários principais: o primeiro representa a gramática, associando cada numeração a uma estrutura que define seu símbolo e produções; o segundo armazena a tabela de parsing, gerada a partir do site “jsmachines” (conforme citado na documentação do trabalho prático).

A análise sintática inicia ao receber a fita de tokens produzida pelo analisador léxico. Com a pilha contendo apenas o estado inicial (0), o analisador começa a percorrer cada token da fita. Para cada token, a tabela de parsing é consultada com base no estado atual e no símbolo da fita, indicando a ação a ser tomada e o próximo estado. As ações possíveis são "shift" (empilhar o novo estado) ou "reduce" (reduzir, seguido de um salto para um novo estado).

Quando a ação é de empilhamento, o próximo estado é adicionado à pilha sem incluir o token lido. No caso de uma redução, a pilha é ajustada removendo símbolos de acordo com o tamanho da produção, e o estado de salto é empilhado em seguida. Se o analisador alcançar o estado “acc”, a fita é aceita, sinalizando que a análise foi bem-sucedida; caso contrário, é exibida uma mensagem de erro indicando o token e a linha onde a falha ocorreu.

4. Ações Semânticas

A análise sintática é uma etapa essencial no processo de compilação, responsável por validar a estrutura de um código-fonte de acordo com as regras gramaticais definidas. Entretanto, apenas a validação estrutural não é suficiente para compilar programas; é necessário atribuir significado ao código analisado. Este processo é realizado pelas ações semânticas**,** que permitem associar lógica e operações às regras gramaticais enquanto a análise sintática ocorre.

Ações semânticas desempenham um papel fundamental em várias tarefas, incluindo a verificação de declarações de variáveis, por exemplo. Estas ações são frequentemente integradas ao analisador sintático e acionadas quando determinadas produções gramaticais são reconhecidas, sendo portanto dirigidas pela sintaxe.

4.1 Implementação

A ação semântica implementada realiza a verificação de definição de variáveis, ou seja, o analisador verifica se as variáveis utilizadas em expressões foram previamente declaradas. Essa funcionalidade opera da seguinte forma:

1. **Identificação de Declarações:**
   Durante as reduções no processo de análise sintática, o analisador verifica se o símbolo que nomeia a regra reduzida é "Decl", representando uma declaração de variável (como na produção número 6 da GLC). Quando uma declaração é identificada, a variável é adicionada à tabela de símbolos, que foi implementada

como um conjunto (*set*), garantindo que as variáveis sejam armazenadas de forma única e eficiente.

2. **Verificação de Uso em Expressões:**
   Ainda na etapa de reduções, o analisador verifica se o símbolo que nomeia a regra reduzida é "Expr", representando uma expressão (produções 7, 8 e 9). Nesses casos, o analisador coleta informações sobre os possíveis operadores da expressão. Os operadores são obtidos a partir da fita de entrada:
   - Para produções como Expr -> Expr + Expr, os dois operadores são extraídos das posições relativas ao índice atual da fita (um operador no índice atual menos um, e o outro no índice menos o tamanho da produção).
   - Para produções como Expr -> NUM, onde há apenas um operador, o mesmo valor será usado duas vezes.

3. **Cada operador é representado como uma tripla contendo:**

   - O número da linha do operador.
   - O tipo (variável ou número).
   - O rótulo associado.

4. **Validação das Variáveis:**
   Após identificar os operadores, o analisador verifica se algum deles é uma variável e, se for, se está presente na tabela de símbolos. Caso a variável não tenha sido declarada anteriormente, um erro semântico é gerado. Nesse cenário, o processo de análise sintática é interrompido antes que o estado de aceitação seja alcançado.

A abordagem escolhida garante que todas as variáveis estejam devidamente declaradas, evitando erros no uso de variáveis não definidas e fortalecendo a integridade semântica do código analisado.

5. Conclusão

O trabalho propôs a implementação de um analisador léxico por um autômato AFD e um sintático SLR com base em uma tabela de parsing gerada pelo site jsmachines e uma ação semântica que verifica a definição prévia de variáveis. A implementação alcançou bons resultados na interpretação da gramática e na identificação de erros. Sendo capaz de identificar tokens corretamente durante a análise léxica, realizar a verificação sintática por meio do analisador sintático, e realizar uma ação semântica dirigida por sintaxe.